

SCHOOL OF COMPUTING (SOC)

Diploma in Applied AI and Analytics

ST1507 DATA STRUCTURES AND ALGORITHMS (AI)

2025/2026 SEMESTER 2

ASSIGNMENT ONE (CA1)

~ Keyword Cipher Encrypted Message Analyzer ~

Objective of Assignment

To practice what you have learnt in the module on data structures, algorithms, and object-oriented programming by developing a Keyword Cipher Encrypted Message Analyzer Application.

Instructions and Guidelines:

1. This is an individual assignment, and it accounts for **30%** of your final grade.
2. The submission date is **Friday 28 November at 1:00 pm**.
3. The development will be carried out in Python using Anaconda. You should only make use of those libraries that already ship with Anaconda Jupyter. You should not make use of the collection library.
4. The interviews will be conducted during the DSAA lessons in week 7. You are expected to explain your code and program logic. Take note that the interview is compulsory.
5. **50% of marks** will be deducted for submission of assignment within **ONE** calendar day after the deadline. **No marks shall** be awarded for assignments submitted **more than one day** after the deadline.

Warning: Plagiarism means passing off as one's own the ideas, works, writings, etc., which belong to another person. In accordance with this definition, you are committing plagiarism if you copy the work of another person and turning it in as your own, even if you would have the permission of that person.

Plagiarism is a serious offence, and if you are found to have committed, aided, and/or abetted the offence of plagiarism, disciplinary action will be taken against you. If you are guilty of plagiarism, you may fail all modules in the semester or even be liable for expulsion.

Context of the application:

A Keyword Cipher is a type of substitution cipher that uses a keyword to create a scrambled alphabet for encryption and decryption. To generate the cipher alphabet, you remove repeated letters from the keyword, write them first, and then append the remaining letters of the alphabet in their original order. Each letter in the plain text is then substituted with the corresponding letter in the newly created cipher alphabet to produce an encrypted text.

Take note, you can read more on substitution ciphers such as Keyword Cipher at Wikipedia.

https://en.wikipedia.org/wiki/Keyword_cipher

Next is an example that demonstrates how a message (plain text) can be encrypted with a Keyword Cipher. When we use the keyword 'ZEBRAS' it will give us the following two alphabets.

Plain Text alphabet
ABCDEFGHIJKLMNOPQRSTUVWXYZ
Cipher Text alphabet
ZEBRASCDFGHIJKLMNOPQTUVWXY

To encrypt each letter in plain text, we use the Cipher Text alphabet to look up with what letter it needs to be replaced.

For instance, with keyword 'ZEBRAS' a plain text message as:

Flee at once, we are discovered!

Will then be encrypted as:

Siaa zq lkba, va zoa rfpbluaoar!

In the same manner we can then also do a decryption using the same keyword 'ZEBRAS'

An encrypted text as:

Siaa zq lkba, va zoa rfpbluaoar!

Will then be decrypted as:

Flee at once, we are discovered!

A Keyword Cipher, just like all substitution ciphers, is not considered to be a strong form of encryption, and is relatively easy to crack, by applying statistical analysis on the frequencies of letters used.

Basic functionality of the application

Your job is to implement a *Keyword Cipher Encrypted Message Analyzer* Application. The application will serve the following three purposes:

1. It will allow a user to encrypt and decrypt text files using a Keyword Cipher.
2. It will allow a user to analyze letter frequencies distributions and then use that to infer an (unknown) keyword from a Keyword Cipher encrypted file.
3. It will allow to batch process multiple Keyword Cipher encrypted files as to infer the keywords that were used to encrypt them.

Take note:

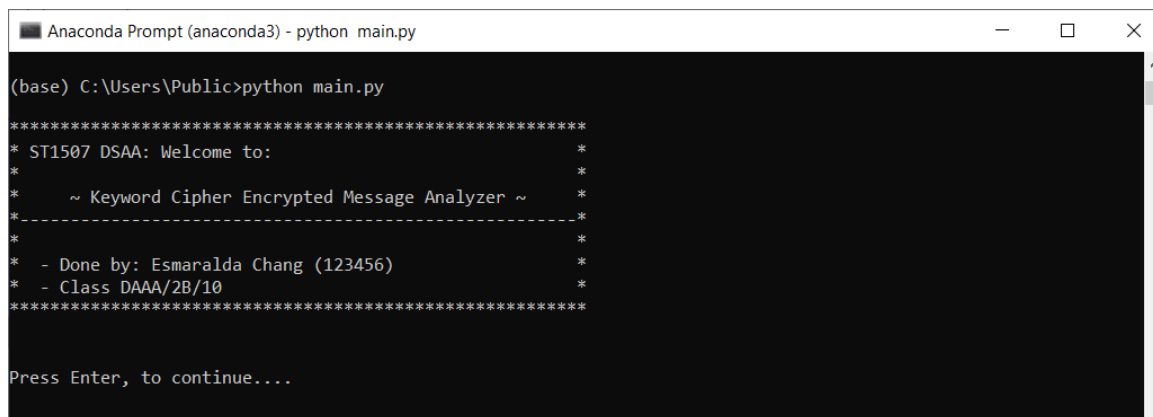
- For our application you may assume that keywords may contain repeated letters. So, for a keyword, 'BUTTERFLY', after removing the repeating letters, we would use 'BUTERFLY' to encrypt.

Starting the application

Your application must be able to start via the Anaconda Command prompt by typing:

```
python main.py.
```

It will then display a title bar with essential information. Thereafter it waits for the user to press the enter key for the application to continue.



```

Anaconda Prompt (anaconda3) - python main.py
(base) C:\Users\Public>python main.py

*****
* ST1507 DSAA: Welcome to:                               *
*                                                         *
* ~ Keyword Cipher Encrypted Message Analyzer ~          *
* -----*
*                                                         *
* - Done by: Esmaralda Chang (123456)                     *
* - Class DAAA/2B/10                                       *
*                                                         *
*****

Press Enter, to continue...
```

- You are required to follow the above format.
- Please ensure that you display your name, student ID and class in the format as is shown in the example.

Selection menu

When the application continues, the user will be presented with a menu as shown below. The menu allows the user to choose from 7 options.

```
Press Enter, to continue....

Please select your choice: (1,2,3,4,5,6,7)
    1. Encrypt/Decrypt File
    2. Letter frequency distribution
    3. Infer cipher keyword
    4. Batch Decryption
    5. Extra Option One
    6. Extra Option Two
    7. Exit
Enter choice:
```

- You are required to follow the above format for the menu display.
- Options 5 and 6 are reserved as extra options that you will need to design and implement yourself.
- The user will be able to repeatedly select the various options from the menu.
- The application will terminate once the user selects option 7, displaying an Exit message as shown below.

```
Press Enter, to continue....

Please select your choice: (1,2,3,4,5,6,7)
    1. Encrypt/Decrypt File
    2. Letter frequency distribution
    3. Infer cipher keyword
    4. Batch Decryption
    5. Extra Option One
    6. Extra Option Two
    7. Exit
Enter choice: 7

Bye, thanks for using ST1507 DSAA: Keyword Cipher Encrypted Message Analyzer
```

Option 1: Encrypt/Decrypt file

The user may select option '1' to encrypt or decrypt a text file. The user will be prompted to choose either 'E' for encryption or 'D' for decryption. Thereafter the user will be required to type the name of both an input file (as to read the plain text from) and an output file (as to write the encrypted text to), followed by a request to type in the cipher keyword.

```
Please select your choice: (1,2,3,4,5,6,7)
  1. Encrypt/Decrypt File
  2. Letter frequency distribution
  3. Infer cipher keyword
  4. Batch Decryption
  5. Extra Option One
  6. Extra Option Two
  7. Exit
Enter choice: 1

Enter "E" for Encrypt or "D" for Decrypt: E

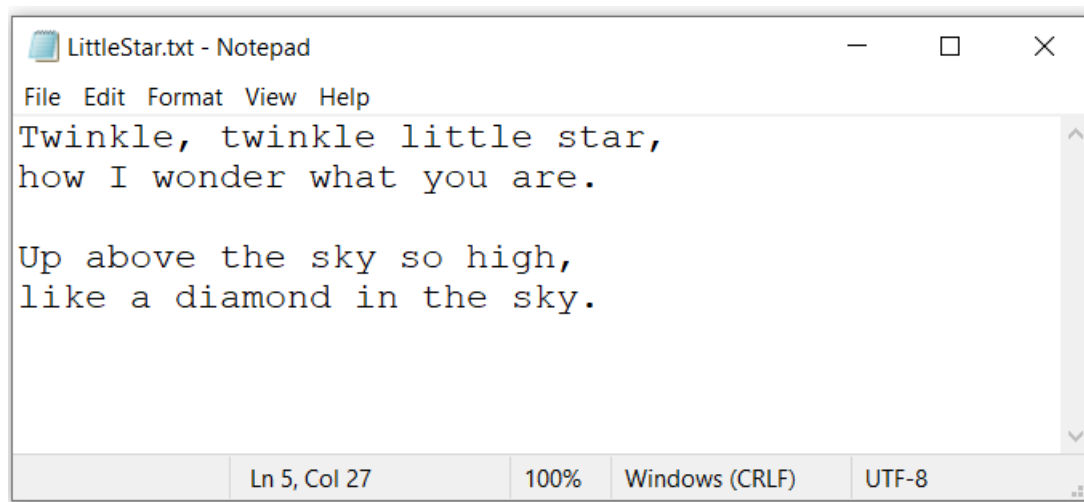
Please enter the file your want to encrypt: LittleStar.txt

Enter the keyword: zebras

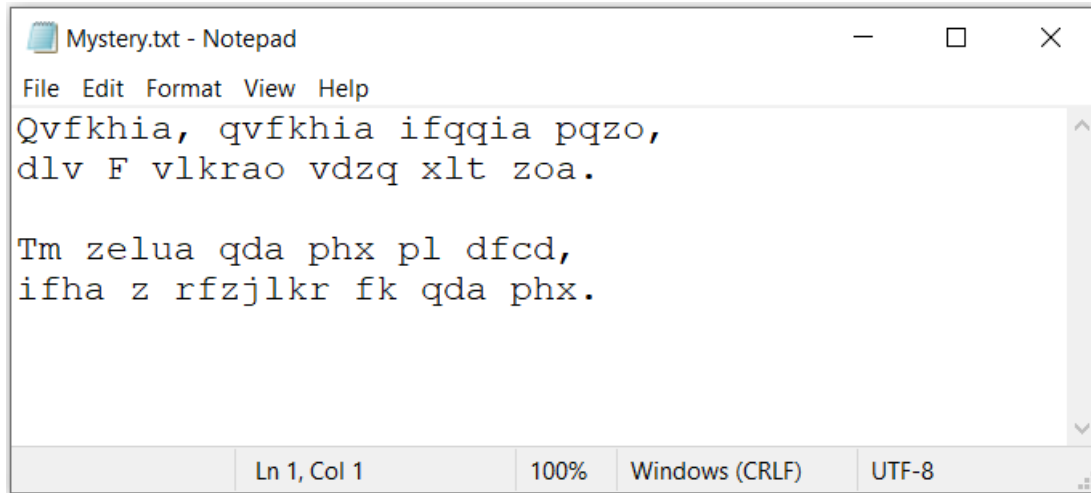
Please enter a output file: Mystery.txt

Press Enter, to continue....
```

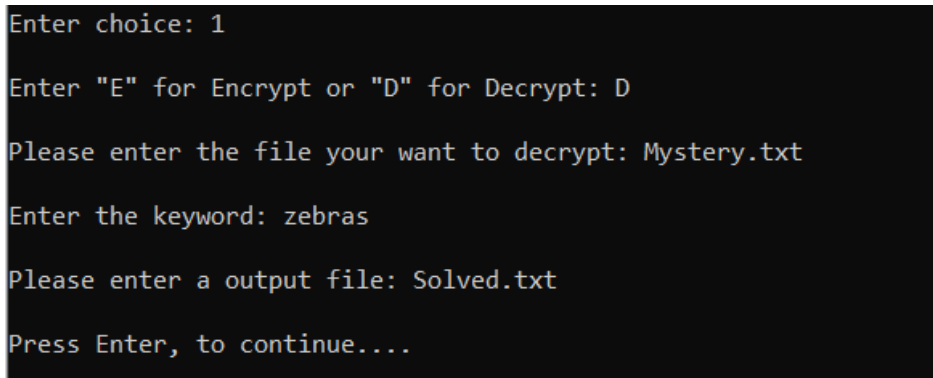
In this example we are doing an encryption with a cipher keyword 'zebras'. The plain text is as follows:



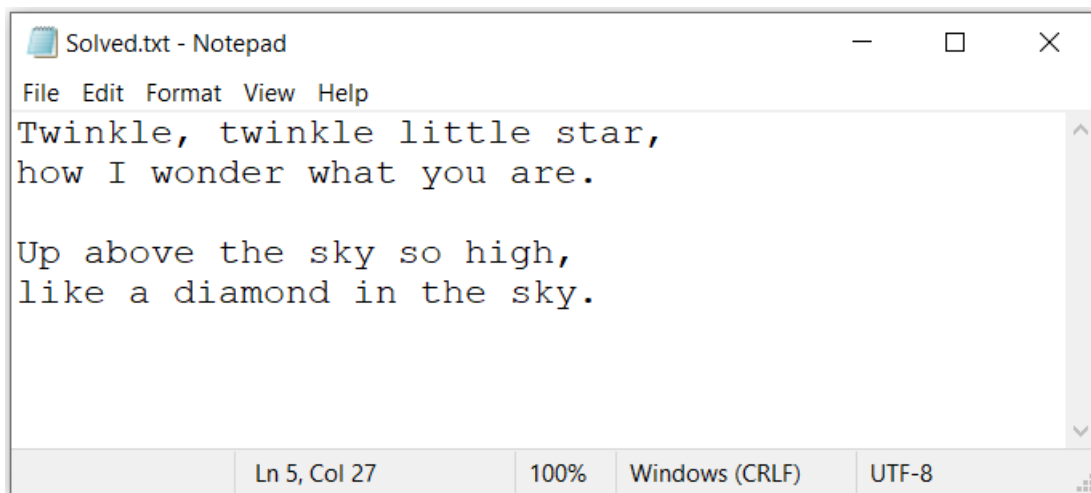
The encrypted text will then be as follows:



We may decrypt the file by using the same keyword of 'zebras', as is shown below.

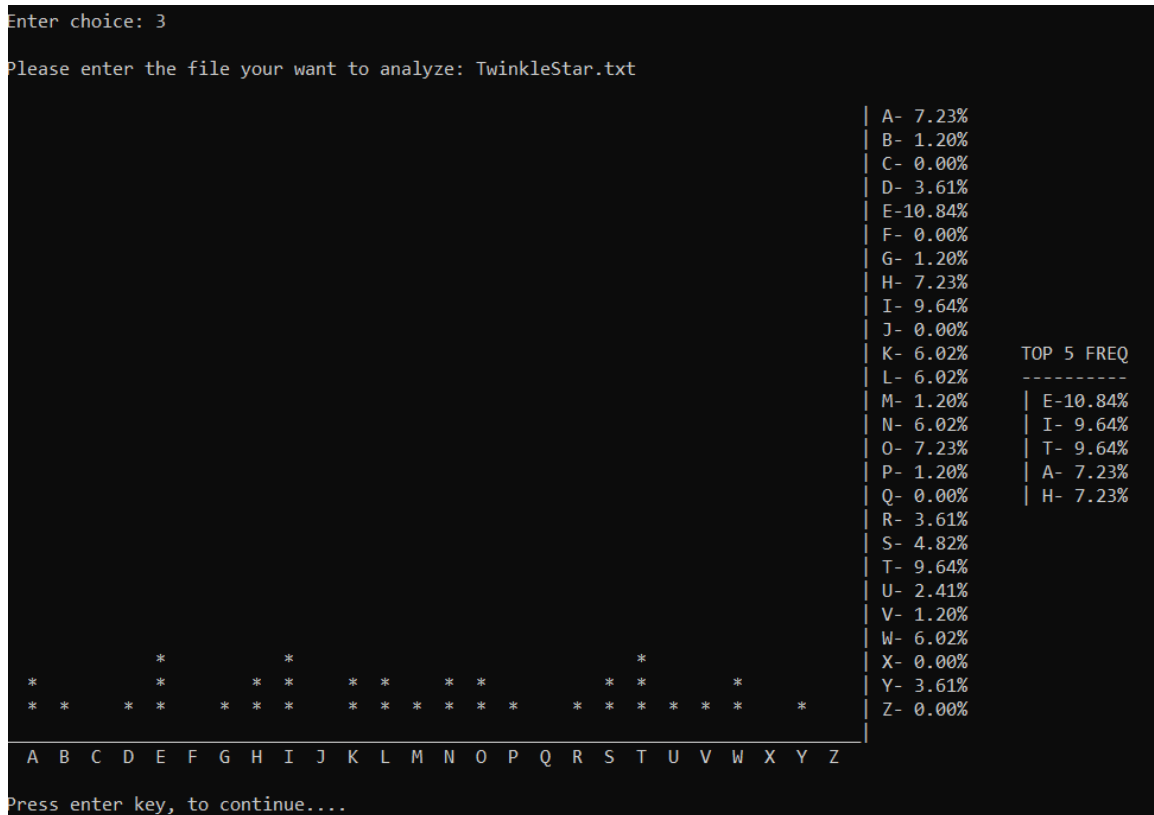


It would give us back the original plain text.

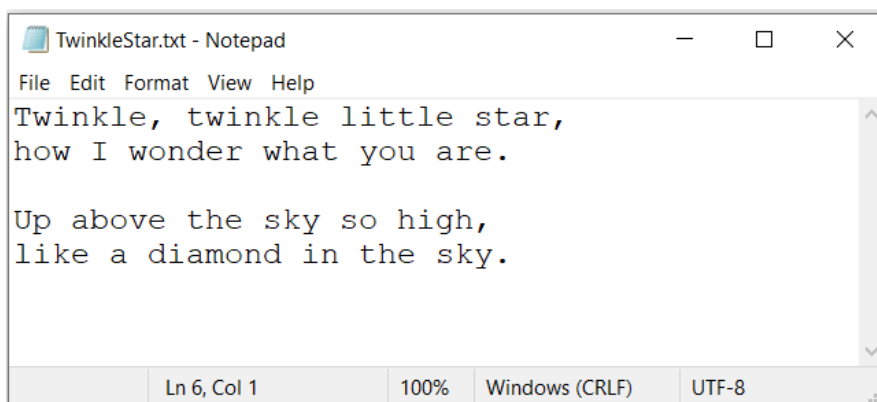


Option 2: Letter frequency distribution

A user can perform letter frequency distribution analyses, by selecting option '2'. After entering the filename, the letter frequencies will be plotted as a bar graph (as shown below). On the right-hand side, it will display the frequency of each letter in the alphabet (rounded to two decimals). Also, the top 5 frequencies, sorted by highest frequency, will be displayed. Take note, that when multiple letters have the same frequencies, they will be sorted by ascending in alphabetical order when listing the top 5 frequencies.



The text that was used for the above displayed letter frequency distribution analysis was as follows:



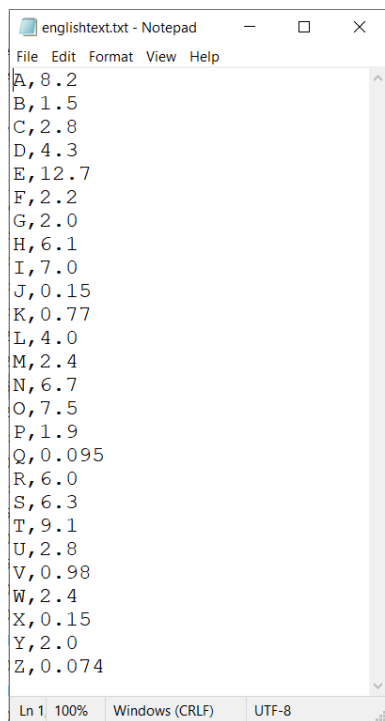
Option 3: Infer Cipher Keyword

By selecting option '3' the user will be able to perform a letter frequency distribution analysis to infer a keyword from a keyword cipher encrypted text. It will thereby make use of the reference letter frequency distribution of English text (provided in Appendix A) and of a list of possible keywords.

In crypto-analysis, frequency analysis is a technique to break simple ciphers such as Keyword cipher for situations whereby the cipher keyword is unknown to you. A Keyword Cipher, as all substitution ciphers, is indeed easily broken by a letter frequency analysis. This is done by analyzing and comparing the letter frequency distribution from the encrypted text with that from the frequency distribution for letters in English text in general (based on averages of large amounts of English text). The letter frequency distribution in English text has a distinct pattern, that is merely rearranged when you do a Keyword Cipher encryption on a text, and hence this would allow you to infer the cipher key with some effort. You may read more on this technique at this Wikipedia site.

https://en.wikipedia.org/wiki/Frequency_analysis

The frequencies for letters in English text (based on average occurrences of letters in large amount English text) is provided in Appendix A. Take note each letter is followed by the percentage of the average occurrence of that letter in English text. Question can you spot the top 5 letters?



A, 8.2
B, 1.5
C, 2.8
D, 4.3
E, 12.7
F, 2.2
G, 2.0
H, 6.1
I, 7.0
J, 0.15
K, 0.77
L, 4.0
M, 2.4
N, 6.7
O, 7.5
P, 1.9
Q, 0.095
R, 6.0
S, 6.3
T, 9.1
U, 2.8
V, 0.98
W, 2.4
X, 0.15
Y, 2.0
Z, 0.074

(*) Source Wikipedia:

https://en.wikipedia.org/wiki/Letter_frequency

Your job is to determine what keyword would produce the best matching letter frequency distribution as with regards to the reference distribution.

After entering the encrypted file, the user will be prompted to provide a list of possible keywords. The application will then proceed by inferring the encryption keyword by means of a letter frequency distribution analysis (thereby using the reference file from Appendix A). Once the keyword has been inferred the user may optionally use that keyword to decrypt the encrypted file.

```
Press Enter, to continue....

Please select your choice: (1,2,3,4,5,6,7)
  1. Encrypt/Decrypt File
  2. Letter frequency distribution
  3. Infer cipher keyword
  4. Batch Decryption
  5. Extra Option One
  6. Extra Option Two
  7. Exit
Enter choice: 3

Please enter the file to analyze: mystery.txt

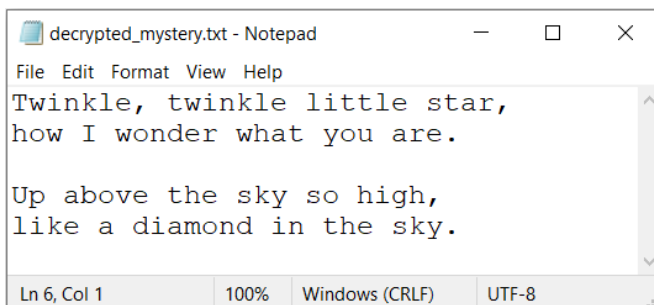
Please enter the keyword file: keyword_list.txt

The inferred keyword is: zebras
Would you want to decrypt this file using this key? y/n: y

Please enter a output file: mystery_zebras.txt

Press Enter, to continue....
```

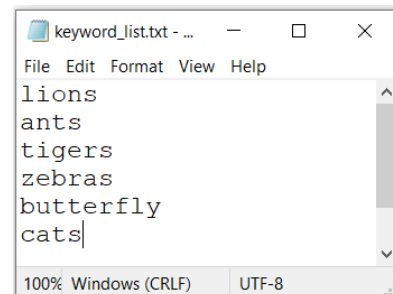
In this case the inferred keyword is 'zebras', and the text will be decrypted to the original plain text.



```
decrypted_mystery.txt - Notepad
File Edit Format View Help
Twinkle, twinkle little star,
how I wonder what you are.

Up above the sky so high,
like a diamond in the sky.

Ln 6, Col 1    100%    Windows (CRLF)    UTF-8
```



```
keyword_list.txt - ...
File Edit Format View Help
lions
ants
tigers
zebras
butterfly
cats

100%    Windows (CRLF)    UTF-8
```

Option 4: Batch Decryption

The user can use option '4' as to do batch processing of multiple encrypted files that are stored in a folder. For instance, a folder may contain the following 5 keyword encrypted files. The files were all encrypted with a keyword that came from a list of possible keywords (you may assume that each keyword in the list was used at the most only once),

Name

 bmjlor.txt
 diqsof.txt
 gnfxiu.txt
 qfzvob.txt
 zenxvy.txt

- Take note, this folder is assumed to be in the same directory where your main.py file is located.

The user will be prompted to enter a folder name, containing the encrypted files as well the filename of a text file with possible keywords.

```
Press Enter, to continue....

Please select your choice: (1,2,3,4,5,6,7)
    1. Encrypt/Decrypt File
    2. Letter frequency distribution
    3. Infer cipher keyword
    4. Batch Decryption
    5. Extra Option One
    6. Extra Option Two
    7. Exit
Enter choice: 4

Please enter the folder name: CASE01

Please enter the keyword file: keyword_list.txt
Decrypting: gnfxiu.txt with keyword: butterfly as: file1.txt

Decrypting: bmjlor.txt with keyword: cats as: file2.txt

Decrypting: qfzvob.txt with keyword: lions as: file3.txt

Decrypting: diqsof.txt with keyword: tigers as: file4.txt

Decrypting: zenxvy.txt with keyword: zebras as: file5.txt

Press Enter, to continue....
```

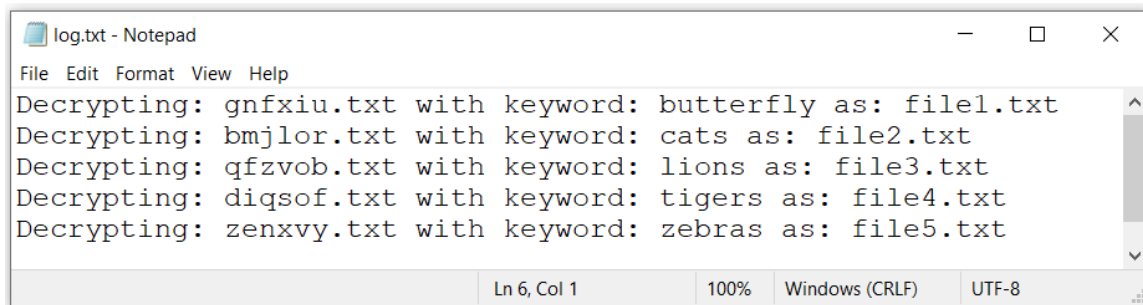
The files in the folder will then be analyzed to infer the encryption keywords and also decrypted with those inferred keywords. The resulting output files will be sorted and named as based on ascending order of the inferred keywords, as file1.txt, file2.txt, file3.txt etc.

After the batch processing of files is completed, the folder will contain the original encrypted files as well as the decrypted files, and a log file (as shown below).

Name

- file1.txt
- file2.txt
- file3.txt
- file4.txt
- file5.txt
- log.txt
- bmjlor.txt
- diqsof.txt
- gnfxiu.txt
- qfzvob.txt
- zenxvy.txt

The log file will indicate for each decrypted file, from what file it was decrypted, and with what keyword. Take note that the filenames for the decrypted files will reflect the order of the keywords in ascending order. Below is an example of what the log file would look like as for this case.



```
log.txt - Notepad
File Edit Format View Help
Decrypting: gnfxiu.txt with keyword: butterfly as: file1.txt
Decrypting: bmjlor.txt with keyword: cats as: file2.txt
Decrypting: qfzvob.txt with keyword: lions as: file3.txt
Decrypting: diqsof.txt with keyword: tigers as: file4.txt
Decrypting: zenxvy.txt with keyword: zebras as: file5.txt
Ln 6, Col 1 100% Windows (CRLF) UTF-8
```

Option 7: Exit

The user can repeatedly select Options 1 till 6. Option 7 is to exit the program. Once the user exits the application you should display the message shown below.

```
Please select your choice: (1,2,3,4,5,6,7)
    1. Encrypt/Decrypt File
    2. Letter frequency distribution
    3. Infer cipher keyword
    4. Batch Decryption
    5. Extra Option One
    6. Extra Option Two
    7. Exit
Enter choice: 7

Bye, thanks for using ST1507 DSAA: Keyword Cipher Encrypted Message Analyzer
```

Basic requirements:

- You are required to design and write the Python application using an object-oriented programming approach (OOP). You should leverage on your knowledge of encapsulation, function/operator overloading, polymorphism, inheritance etc.
- You may make use of Python's standard data structures, such as list, tuple, dictionary etc., however you should refrain from using the classes from the collection library. Instead, you are required to write your own classes to support the various data structures that you may need (for instance for sorting you may need a class *SortedList*). Of course, you may refer to the lecture slides and lab tasks and expand further on those classes that we had previously developed and discussed in tutorials and lab sessions.
- To run the application there should be no need to install additional libraries, other than those that ship already with Anaconda.
- Your application should not have to rely on any connection to the Internet.
- The OOP classes that you develop must be placed in separate python files.
- Pay attention to user input validation. Your application should not crash if a user types in the wrong input. Instead, when a user enters wrong input, you should notify the user and allow him/her to enter again.

Deliverables

Your deliverables must include:

(a) Source Code

- You must submit all the python files (py files) that make up your application. Ensure the code is complete, and that it can run directly from the Anaconda Prompt as:

```
python main.py.
```

(b) Report

A report (as pdf file) with a maximum of **8** pages. This would exclude the cover page and the appendix with the source code and references. The report should contain:

- a) Cover page with title and your name, student ID and class.
- b) Description, and user guidelines, on how to operate your application (please include screen shots of your application in action).
- c) Description of how you have made use of the Object-Oriented Programming (OOP) approach. You may elaborate on the classes that you have developed, and discuss issues such as encapsulation, function/operator overloading, polymorphism, inheritance etc. Do include a class diagram that displays the relation between the various classes that you have developed.
- d) Discussion on the data structures and algorithms that you have developed for your application. You may discuss issues such as, the performance of the algorithms in terms of Big (O). Do explain why you did develop certain data structures and explain why you deem these data structures suitable for the tasks at hand. Do include a table summarizing all the data structures that you have been using (those that you have developed and those already built in Python).
- e) Include a summary of the challenges that you have faced while developing the application. Provide a summary of the key takeaways and learning achievements that you have obtained from this project.
- f) **All** your python **source code** must be included as an appendix at the end of your report.

(c) Academic Integrity Declaration form

- Ensure that the form is duly filled in and signed.

Submission requirements

- Submit all the deliverables (Source Code, test files, Academic Integrity Declaration form) in the designated BrightSpace drop box before the submission deadline.
- You must submit it as one Zipped folder (RAR will not be accepted, only zip).

Label your submission as:

CA1_Name_StudentID_Class.zip

For example: *CA1_EsmaraldaChang_12345_2B10.zip*

Assessment Criteria

The assignment will be assessed based on the following criteria:

Assessment criteria	Marks awarded
User Interface - GUI looks and operates as prescribed. - GUI supports appropriate user input validation and error handling	Max 10
Basic functionality of the application: Option 1 - Able to encrypt a text file with a keyword cipher. - Able to decrypt a text file with a keyword cipher. Option 2 - Able to extract letter frequencies distribution. - Able to display letter frequencies distributions in the required format. Option 3/4 - Able to infer a keyword from an encrypted file. - Able to batch process the inferring of keywords from encrypted files.	Max 30
Two Extra Features (Option 5 & 6): - Features will be judged according to technical sophistication and useability of features.	Max 20
Programming techniques, efficiency, robustness, and readability of code: - Appropriate usage of classes and OOP technology. - Appropriate usage of data structures and algorithms. - Code is properly commented and neatly structured. - Application is free of crashes.	Max 20
Report: - The report follows the prescribed format. - The report is well written and comprehensive,	Max 10
Interview/Demonstration: - Ability to demonstrate and explain the application and the code clearly. - Q&A (*) Multiplier on total score may be applied if student demonstrates poor understanding of code.	Max 10
Grand Total	100

APPENDIX A: Letter Frequency Distribution for English Text

english_letter_frequency.txt
A, 8.2 B, 1.5 C, 2.8 D, 4.3 E, 12.7 F, 2.2 G, 2.0 H, 6.1 I, 7.0 J, 0.15 K, 0.77 L, 4.0 M, 2.4 N, 6.7 O, 7.5 P, 1.9 Q, 0.095 R, 6.0 S, 6.3 T, 9.1 U, 2.8 V, 0.98 W, 2.4 X, 0.15 Y, 2.0 Z, 0.074

-- The End --